MODULE 04: DATA MANIPULATION

1. Introduction to Data Manipulation

In the previous modules, you learned how to create database structures, tables, and data types. Now, we will learn how to add, change, and remove actual data rows inside those tables. This is done using **DML (Data Manipulation Language)** commands. As a data analyst, while you mostly read data, understanding how data is inserted, updated, and deleted is crucial for clean data handling, data cleaning tasks, and preparing mock data sets for testing dashboards.

2. INSERT INTO Command

Simple Explanation

The `INSERT INTO` command is used to add new rows of data into an existing table. Think of it like adding a completely new row of information at the bottom of an Excel sheet.

Exam Definition

INSERT INTO: A Data Manipulation Language (DML) statement utilized to append one or more new record rows into a specific database table structure without altering the existing schema attributes.

Standard Syntax

```
INSERT INTO table_name (column1, column2, column3)
VALUES (value1, value2, value3);
```

Practical Example

Let's say we have an empty `customers` table with columns: `customer_id`, `customer_name`, and `city`.

```
INSERT INTO customers (customer_id, customer_name, city)
VALUES (1, 'Amit Sharma', 'Mumbai');
```

3. Inserting Multiple Records

Simple Explanation

Instead of writing a separate `INSERT INTO` command for every single row, SQL allows you to add multiple rows all at once inside a single query. You just separate the values with commas. This saves a lot of time and makes the query run much faster.

Exam Definition

Multi-row Insertion: An optimized SQL data entry technique where multiple sets of value parameters, wrapped in parentheses and separated by commas, are supplied to a single INSERT statement to construct multiple record blocks instantly.

Standard Syntax

```
INSERT INTO table_name (column1, column2, column3)
VALUES
(value1_a, value2_a, value3_a),
(value1_b, value2_b, value3_b),
(value1_c, value2_c, value3_c);
```

Practical Example

Adding three new customers to our table in one single go:

```
INSERT INTO customers (customer_id, customer_name, city)
VALUES
(2, 'Priya Nair', 'Bengaluru'),
(3, 'Rohan Das', 'Mumbai'),
(4, 'Sneha Reddy', 'Hyderabad');
```

4. UPDATE Command

Simple Explanation

The UPDATE command is used to edit or modify data rows that are already present in your table. Think of it like clicking on a specific cell in an Excel sheet and changing the typed information.

Exam Definition

UPDATE: A DML statement used to modify existing record values within a table column, either globally across the entire table or selectively targeting particular row partitions based on a specified filtering criterion.

Standard Syntax

```
UPDATE table_name
SET column1 = value1, column2 = value2
WHERE condition;
```

Practical Example

If customer Rohan Das moves from Mumbai to Pune, we change his city value like this:

```
UPDATE customers
SET city = 'Pune'
WHERE customer_id = 3;
```

5. DELETE Command

Simple Explanation

The DELETE command is used to remove existing data rows completely from a table. Think of it like selecting a row in Excel and clicking "Delete Row". The table and its columns stay exactly where they are, but the rows inside disappear.

Exam Definition

DELETE FROM: A DML statement utilized to permanently remove individual or multiple row records from a database table based on a filtering conditional clause, while preserving the structural integrity and constraints of the empty grid.

Standard Syntax

```
DELETE FROM table_name
WHERE condition;
```

Practical Example

If customer number 4 leaves our platform, we remove her record row from the table:

```
DELETE FROM customers
WHERE customer_id = 4;
```

6. The Critical Importance of WHERE with UPDATE and DELETE

> ⚠️ **DANGER ZONE: The Golden Rule for Analysts & Engineers**
> Always remember: The `WHERE` clause tells SQL *which specific row* you want to change or delete. If you forget to write the `WHERE` clause in an UPDATE or DELETE command, **SQL will apply the change to every single row in the entire table!**

Let's look at what happens if you make this mistake in an interview or a real job database:

The Mistake with UPDATE

```
-- WRONG WORK: Missing the WHERE clause!
UPDATE customers
SET city = 'Pune';
```

Result: Every single customer in your entire database will now have their city changed to 'Pune'. All your original data is overwritten and destroyed.

The Mistake with DELETE

```
-- WRONG WORK: Missing the WHERE clause!
DELETE FROM customers;
```

Result: Every single row inside the table is wiped clean. You will end up with a completely blank table, losing thousands of historical customer records instantly.

7. Command Summary Quick-Reference Matrix

Command Name	Type	Main Purpose	Requires WHERE for Safety?	Impact on Table Structure
INSERT INTO	DML	Adds completely new rows into a table.	No	None (Structure remains unchanged).
UPDATE	DML	Modifies existing cell values in matching rows.	**YES!** (Otherwise modifies all rows).	None (Structure remains unchanged).
DELETE FROM	DML	Removes specific row items from a table.	**YES!** (Otherwise empties the whole table).	None (Structure remains unchanged).

8. Real-World Analyst Case Study & Solution

Interview Scenario: You are working as a junior data analyst at a global fintech startup. The marketing operations team accidentally pushed an invalid discount script into production. Because of this glitch, any customer residing in the city of **'New Delhi'** who signed up today was mistakenly given an `account_balance` value of `999999` instead of their actual opening balance of `500`. Furthermore, test user accounts with the email domain `'@testbot.com'` were mistakenly inserted and need to be permanently purged before metrics calculations begin.

Your Task: Write a safe data repair script that accomplishes two things:

1. Fix the corrupted balance values for the customers in 'New Delhi' without touching other fields.
2. Permanently purge the robotic test accounts to ensure clean metric outputs.

Structured Query Solution

```
-- STEP 1: Repair the corrupted balance data rows safely using exact filters
UPDATE user_accounts
SET account_balance = 500
WHERE city = 'New Delhi' AND account_balance = 999999;

-- STEP 2: Wipe out the dirty robot test user records securely
DELETE FROM user_accounts
WHERE email LIKE '%@testbot.com';
```

Interview Technical Justification Matrix

Action Taken	SQL Element Used	Analytical Logic For Interview Panels
Targeted Data Fix	`UPDATE ... SET ... WHERE`	Ensures only the broken balance fields for New Delhi users are overwritten. Leaving out the balance filter could accidentally override valid high-value clients in Delhi.
Data Cleaning / Purging	`DELETE FROM ... WHERE ... LIKE`	Isolates and wipes out test records using wildcards (%). This prevents skewed customer metric counts while keeping the baseline infrastructure completely clean.